Credit Card Fraud Detection System

Comprehensive Class Presentation, Technical Defense & Demonstration Guide

1. Project Title & 30-Second Elevator Pitch (Opening Slide)

Speaker Pitch:
"Good morning everyone. Today, I am presenting our Machine Learning-Based Credit Card Fraud Detection Platform. Every single day, billions of dollars move across digital payment rails. Banks face a critical challenge: a single missed fraud causes hundreds of euros in direct chargeback losses, while too many false alarms alienate honest customers with rejected cards. In this project, we built an end-to-end production-grade system that resolves severe class imbalance (0.17% fraud rate), benchmarks multiple algorithms, calibrates decision thresholds against real financial cost curves, and exposes live predictions through a reactive fintech dashboard."

2. The Core Problem: Why Standard ML Fails

When presenting the problem statement, emphasize two foundational hurdles:

A. The Extreme Class Imbalance Trap: Our real-world European cardholder dataset contains **284,807 transactions**, but only **492 are fraudulent (0.172%)** — roughly 1 fraud per 578 transactions. If a naive model simply predicts 'Genuine (0)' for every transaction, it scores **99.83% Accuracy** while catching exactly 0 frauds. Thus, standard Accuracy is entirely useless; models must be judged on **PR-AUC**, **Recall**, and **F2-Score**.

B. Business Asymmetry (Cost of Errors): A False Negative (missed fraud) costs €250+ in direct unrecovered chargeback losses, whereas a False Positive (false alarm) costs only €10–€20 for an automated SMS verification or customer check. Catching fraud is 10× to 25× more critical than avoiding minor verification checks.

3. End-to-End Machine Learning Pipeline

Our pipeline is structured into five distinct, leakage-free phases:

• **Phase 1: Preprocessing & Leakage-Free Stratification**: The dataset contains 30 numerical features: Time, Amount, and V1–V28 (PCA transformed for PCI-DSS data privacy). Skewed transaction amounts were normalized using Log transformation (log1p(Amount)) and RobustScaler. Data was split into 85% Train / 15% Test using stratified sampling to preserve the 0.172% fraud distribution in both splits without data leakage.

• **Phase 2: Handling Imbalance (SMOTE vs. Cost Weighting)**: We compared two industry paradigms: (1) Data-Level SMOTE (Synthetic Minority Over-sampling Technique) applied strictly to the training split, and (2) Algorithm-Level Cost-Sensitive Weighting (scale_pos_weight in XGBoost, balanced class weights in RF and LR).

• **Phase 3: Multi-Algorithm Benchmarking**: Six model configurations were trained: Logistic Regression (Weighted & SMOTE), Random Forest (Weighted & SMOTE), XGBoost (Weighted & SMOTE), and a Hyperparameter-Tuned Champion XGBoost.

• **Phase 4: Cost-Sensitive Threshold Optimization**: Rather than accepting the naive default threshold ($\theta = 0.50$), we computed a financial expected cost curve across $\theta \in [0.01, 0.95]$ to find the exact threshold minimizing operational bank losses.

• **Phase 5: Explainability & Full-Stack Deployment**: Utilized SHAP (SHapley Additive exPlanations) to isolate top fraud drivers (V14, V17, V12, V10, V4, V11) and deployed the champion pipeline via FastAPI with a reactive

5-tab HTML5/JS dashboard.

4. Benchmark Results & Key Findings

All models were evaluated on the held-out test set (42,721 transactions, including 71 actual frauds):

Algorithm	PR-AUC (Primary)	ROC-AUC	Recall (Catch Rate)	Precision	False Positives	False Negatives
■ **Tuned XGBoost (Champion)**	**0.8527**	0.9845	**82.97%**	**81.69%**	**13**	**12**
Random Forest + SMOTE	0.8485	0.9865	83.55%	79.80%	15	12
XGBoost + SMOTE	0.8285	0.9843	78.33%	81.10%	13	15
XGBoost (Cost Weighted)	0.8145	0.9744	70.69%	81.20%	11	21
Random Forest (Class Weighted)	0.8098	0.9720	81.95%	78.50%	16	13
Logistic Regression + SMOTE	0.7410	0.9818	84.50%	12.50%	422	11

Key Presentation Takeaways:

1. **Why Linear Models Fail:** Logistic Regression achieved 84.5% Recall, but had terrible Precision (12.5%), triggering **422 false alarms** on test data alone.
2. **Why Tuned XGBoost Won:** Achieved the highest PR-AUC (**0.8527**) and struck the perfect balance with **82.97% Recall**, **81.69% Precision**, and only **13 False Positives** across 42,000+ transactions.

5. The Secret Weapon: Cost-Optimal Threshold Tuning

Standard classification tools default to θ **= 0.50**. In fraud detection, this creates massive losses because subtle fraudsters score between 15% and 45% risk. We formalized the bank's total expected financial loss function:

Total Financial Loss = (False Negatives × €250) + (False Positives × €20)

• **At Default** θ **= 0.50:** Misses borderline frauds, driving up expensive €250 chargeback losses.
• **At Optimal** θ **= 0.12:** Minimizes overall financial cost, lifting fraud catch rate to **over 89%** while keeping false alarm verification costs negligible.

6. Model Explainability & Key Fraud Drivers

Using SHAP (SHapley Additive exPlanations) and statistical distribution delta analysis, we demystified the black box:

• **Major Negative Fraud Drivers (V14, V17, V12, V10):** Normal transactions average near 0.0. During fraud, these features show massive negative spikes (e.g. V14 drops below -5.0), raising fraud probability above 80%.

• **Major Positive Fraud Drivers (V4, V11, V2):** Exhibit abnormal positive surges during coordinated botnet attacks.

7. Live Screen-Share Demonstration Script

Follow this exact 4-step sequence during your presentation:

• **Step 1: Tab 1 (EDA)**: Show the Class Imbalance donut chart (0.17% fraud) and the Feature Shift chart to establish why standard metrics fail.

• **Step 2: Tab 2 (Benchmarking)**: In the sidebar, switch from Tuned XGBoost to Logistic Regression. Point to the Confusion Matrix: False Positives instantly explode from 13 to 422! Switch back to XGBoost to show it dropping to 13.

• **Step 3: Tab 3 (Real-Time Screener)**: Click 'Preset: Borderline Edge Case (€420, Risk ~32%)'. Click 'Optimal θ = 0.12' in the sidebar: The banner turns RED (BLOCKED). Then click 'Default θ = 0.50': The banner turns GREEN (APPROVED). Explain: 'This proves why a 0.12 cost threshold stops fraud that a naive 0.50 threshold lets through!'

• **Step 4: Tab 4 & 5 (Batch & Cost Simulator)**: In Tab 4, click 'Quick Load 50 Transactions' and move the threshold slider to show live batch triage. In Tab 5, increase FN cost to €400 and show the optimal threshold line automatically adjust.

8. Likely Professor & Class Defense Questions

Q1: Why did you prioritize PR-AUC over ROC-AUC?

Answer: In a dataset with 284,000 genuine transactions, True Negatives (TN) are overwhelmingly large. ROC-AUC uses False Positive Rate (FP / [FP + TN]). Because TN is huge, FPR remains artificially near zero, inflating ROC-AUC to 0.98+ even for bad models. PR-AUC measures Precision against Recall without including TN, giving an honest, uninflated measure of minority fraud detection.

Q2: Why must SMOTE only be applied to the training set?

Answer: SMOTE synthesizes examples using k-Nearest Neighbors. Applying SMOTE before splitting allows test set information to leak into training neighbors, causing data leakage and artificially inflated scores that fail when deployed in production.

Q3: Why XGBoost over Deep Neural Networks for this dataset?

Answer: On tabular financial data with structured numerical features, tree-based gradient boosting models (XGBoost/LightGBM) consistently beat deep learning architectures in empirical benchmarks, train in minutes rather than hours, require far less compute, and allow direct feature attribution through tree importance and SHAP values.

Q4: How would this run in real-time bank operations?

Answer: The FastAPI microservice is deployed as a containerized service. When a user swipes a card, an event message is published to Kafka. FastAPI scores the transaction vector in under 5 milliseconds and responds with APPROVE, STEP-UP 2FA, or BLOCK before the physical POS payment terminal completes authorization.